Transcript

Week 16: Neural Network Basics

Video 1 - Neural Network

What if your laptop had a brain of its own? Neural networks are the reason it almost does—let's break it down.

Neural networks are a key part of machine learning. They're inspired by how our brains process information. Just like neurons in the brain pass messages to each other, neural networks use connected layers of artificial "neurons" or nodes. Each node handles data and passes it on, helping the network find patterns. Over time, this helps the network learn from data and make smart predictions.

Biological neurons are the brain's communication units. Each one receives, processes, and sends information to others. A typical neuron has three main parts: dendrites that receive signals, an axon that sends signals, and synapses that pass signals between neurons. This structure inspired the design of artificial neural networks used in machine learning.

Artificial neurons are designed to behave like simplified brain cells. Each one receives inputs, processes them, and produces an output. The connections between them are called weights—they control how important each input is. A bias adjusts the output before it moves to the next layer. Finally, the activation function decides if the neuron should activate, just like a biological neuron fires a signal. Together, these components allow the network to learn from data, adjust over time, and make better predictions—just like our brains learn from experience.

Biological and artificial neurons may look different, but they share important similarities that help explain how neural networks work.

Let's start with **structure**. Biological neurons are made up of dendrites, an axon, and synapses. These parts allow them to receive signals, transmit them, and connect to other neurons. In artificial neurons, the same roles are played by inputs, weights, and activation functions.

In terms of **function**, both types of neurons follow the same basic logic: they take in signals, process them, and produce an output.

Connections are another similarity. Biological neurons communicate through synapses, while artificial neurons are linked by weighted connections that carry signals between layers.

The **learning mechanism** is where things get really interesting. In the brain, learning happens when synaptic strength changes based on experience. In artificial networks, learning means adjusting the weights through a process called backpropagation, based on training data.

Their **organisation** also mirrors each other. Biological neurons are arranged in highly complex networks. Artificial neurons are structured into layers—an input layer, one or more hidden layers, and an output layer.

Finally, let's talk about **activation**. A biological neuron fires only when its input reaches a certain threshold. Similarly, an artificial neuron uses an activation function to decide whether to pass its signal to the next layer.

Together, these similarities show how artificial neural networks are inspired by the brain—and why they're so powerful for learning from data.

In an artificial neural network, the basic unit is a neuron, also called a node or perceptron. Each neuron receives input values, multiplies them by weights, and adds them together. This total is called the weighted sum. The neuron also adds a constant value known as the bias, which helps shift the activation threshold and improves flexibility in learning different patterns.

The inputs, often written as x, are the features or attributes of a particular instance—such as height, age, or colour—and each input gives valuable information for processing. The weights, written as w, are values the neuron learns during training. They determine how strong each input is, helping the model prioritise some features over others.

The bias, written as b, is a fixed value added to the weighted sum. It allows the neuron to shift its output range, improving how the model fits data. After calculating the weighted sum plus bias, the result passes through an activation function, written as f. This function introduces non-linearity, which is crucial for recognising complex patterns.

Common activation functions include the sigmoid, Rel-you—spelled R-E-L-U, meaning Rectified Linear Unit—and tan-h, short for hyperbolic tangent. Each has its own effect on how the neuron responds to inputs.

Together, the inputs, weights, bias, and activation function enable neurons to process information, learn from data, and help the network make accurate predictions or decisions.

Each neuron in a neural network receives data inputs, also called features. These are multiplied by weights, which show how important each input is. The results are added together, along with a constant called the bias. This total is known as the weighted sum.

The formula is: *x one times w one, plus x two times w two, plus b*.

This weighted sum passes through an **activation function**, which decides whether the neuron will respond. Common activation functions include the **sigmoid**, **Rel-you**—spelled R-E-L-U—and the **hyperbolic tangent**.

If the signal passes the threshold, the neuron fires and passes its output to the next layer. This helps the network learn patterns and make predictions from the data.

A neuron in a neural network receives input values, which could be features from a dataset or signals from the previous layer. These inputs are written as x one, x two, and so on. Each input is multiplied by a weight—w one, w two, and so on—which determines how important it is. These weights are learned during training.

The neuron adds all the weighted inputs and a constant value called the bias, written as b. This bias helps shift the output and allows the model to fit more complex patterns. The total calculation is x one times w one, plus x two times w two, plus b.

Next, this sum is passed through an activation function, written as f, which adds non-linearity to the model. This step helps the network handle complex patterns. Common activation functions include sigmoid, Rel-you—spelled R-E-L-U—and tanh.

Finally, the neuron sends the output to the next layer. This process enables the network to learn from data and make decisions.

In Step 2, the neuron receives several input values, written as x one, x two, and so on. These inputs can come from a dataset or a previous layer in the network. Each one represents a specific feature—like age, temperature, or price—and carries useful information for the neuron to process in the next step.

To calculate the weighted sum in a neuron, each input value—written as x one, x two,

and so on—is multiplied by its corresponding weight, w one, w two, and so on. These weighted inputs are then added together.

Next, a constant value called the bias—written as b—is added to the sum. This gives us the final weighted sum, written as z.

The formula is:
z equals x one times w one, plus x two times w two, and so on, plus b.

This value, z, is then passed through an activation function, which helps decide if the neuron should produce an output.

In Step 4, the neuron passes the weighted sum—written as z—into an activation function. Each function shapes how the neuron responds to inputs and determines whether it should activate.

In Step 5, the output of the neuron is calculated as f of z. This value represents the neuron's response and can be interpreted as a signal or prediction.

The output is then sent to the next layer of neurons, or, if it's the final layer, it becomes the model's result.

In Step 6, we measure how far the neuron's predicted output is from the actual target. This is done using a **loss function**, which calculates the error or difference between the predicted value and the true value.

For regression tasks, we often use **Mean Squared Error**, or **M-S-E**, which averages the squared differences between predicted and actual values. Larger errors are penalised more, helping the model minimise big mistakes.

For classification tasks, we use **cross-entropy loss**, which compares predicted probabilities with the actual class. It penalises incorrect predictions, especially when the model is very confident but wrong.

These loss values help the neural network learn—by adjusting internal parameters like weights and biases—to improve future predictions.

In Step 7, we use **Gradient Descent** to reduce error and improve predictions. Gradient Descent is an optimisation method that updates the network's weights to minimise the loss.

First, we calculate the **gradient** of the loss with respect to each weight. This shows how much the loss would change if we slightly adjusted a weight.

Next, we update the weights in the opposite direction of the gradient to reduce the error. The formula is:
w new = w old minus eta times the gradient of the loss with respect to w

Here, eta is the **learning rate**, which controls how large each update step is. If η is too small, the process is slow. If it's too large, the model might become unstable.

This update is repeated through multiple iterations, helping the model gradually improve with each pass over the data.

In Step 8, we use a process called **Backpropagation** to update the network's weights based on the error. This involves sending the error backwards through the network, starting at the output layer.

The algorithm checks how much each weight in earlier layers contributed to the error.

Then, using the gradients calculated during gradient descent, it updates each weight to reduce that error.

This happens layer by layer, moving backwards through the entire network. The process continues until all the weights have been updated. Over time, this helps the network reduce its overall error and make better predictions.

If a single wrong prediction can reshape an entire network—what does that say about how machines learn? Take a moment to connect the dots: how does error travel backwards, influence weights, and ultimately drive smarter decisions?

Video 2 - Activation Functions

What makes a neuron decide to fire—or stay silent? Activation functions add that spark. Let's see how they bring neural networks to life.

Activation functions are essential in deep learning. They decide the output of a neuron by transforming the weighted sum of inputs into a meaningful value.

This transformation adds non-linearity, allowing the network to recognise complex patterns that linear models cannot detect. Without activation functions, neural networks would only solve simple, linear problems.

They're especially important for tasks like image classification, language processing, and other real-world applications where patterns aren't obvious at first glance.

Activation functions are divided into two main types: **linear** and **non-linear**.

Linear activation functions create a straight-line relationship between input and output, but this limits the network's ability to handle complexity.

Non-linear activation functions add flexibility. They allow neural networks to model more advanced patterns—essential for solving real-world problems like image recognition and natural language processing.

The sigmoid activation function is a popular non-linear function used in neural networks. It compresses input values into a range between zero and one.

This makes it ideal for binary classification tasks, especially in the output layer—where the result is interpreted as the probability of belonging to a class.

The formula for sigmoid is: **sigma of x equals one divided by one plus e to the power of minus x**.

Because it's smooth and differentiable, sigmoid works well with gradient-based optimisation. But it does have a limitation—it's prone to the vanishing gradient problem, which can slow down learning in deeper networks.

The tanh activation function is another non-linear function used in neural networks. It transforms inputs into a range between minus one and one.

Tanh is similar to sigmoid but has one key advantage—it is zero-centred. This helps with optimisation by keeping the outputs more balanced around zero.

Its formula is: **tanh of x equals e to the x minus e to the power of minus x, divided by e to the x plus e to the power of minus x.**

Because of its range and balance, tanh is often preferred over sigmoid in hidden layers, especially in deep networks. It supports better gradient flow and more efficient learning.

Rel-you, or Rectified Linear Unit, is one of the most popular activation functions in deep learning. It outputs the input directly when it's positive, and zero when it's not.

Its formula is: **f of x equals the maximum of zero and x.**

Rel-you is fast and easy to compute, which helps speed up training. That's why it's often used in hidden layers of deep networks.

However, there's a limitation known as the "dying Rel-you" problem—some neurons may stop learning if they always return zero.

Leaky Rel-you is a variation of the standard Rel-you function. It's designed to fix the "dying Rel-you" problem, where some neurons become inactive and stop learning.

Instead of outputting zero for all negative inputs, Leaky Rel-you lets a small negative value pass through. This means neurons stay active—even with negative input.

The formula is: **f of x equals x if x is greater than zero, and alpha times x if x is less than or equal to zero**, where alpha is usually a small constant like zero point zero one.

This small tweak improves learning in deeper networks by maintaining a steady flow of gradients during training.

The Softmax activation function helps neural networks handle multi-class classification problems. It builds on the sigmoid function but works with multiple classes by converting raw output values—called logits—into probabilities. Each value shows how likely the input belongs to a specific class.

The formula for Softmax is:
sigma of x-i equals e raised to the power of x-i, divided by the sum of e raised to the power of x-j for all j from 1 to n
This ensures that all output values are between zero and one, and together they add up to one.

Softmax allows the model to make clear, interpretable predictions by highlighting the most likely class based on the input data.

In deep learning, choosing the right activation function is key to how well a neural network learns. Each function shapes how data flows through the network and how effectively the model trains.

The **Sigmoid** function outputs values between zero and one, making it useful in binary classification—but it can slow down learning in deep networks.
Tan-h ranges from minus one to one. It's zero-centred and generally preferred over Sigmoid in hidden layers.
Rel-you outputs zero or higher. It's fast and effective, but some neurons can stop learning—this is known as the dying Rel-you problem.
Leaky Rel-you solves this by letting small negative values through, with an output range that includes negative and positive values.
Softmax transforms a set of outputs into probabilities that add up to one, making it perfect for multi-class classification.

By understanding both the behaviours and the output ranges of these functions, we can choose the right one for the task at hand.

Choosing the right activation function is like picking the right study method—it depends on the task. Which one fits your model's goals best?

Video 3 - Loss Functions

Ever submitted an assignment and instantly knew you got something wrong? That's exactly what a loss function does—it helps the model figure out how far off it was so it can do better next time.

In deep learning, loss functions help the model learn by measuring how far off its predictions are from the actual results. Think of them as a guide that tells the model what it got wrong. This feedback helps the model improve over time.

By reducing the loss, the model makes better predictions. A good loss function not only improves accuracy, but also helps the model generalise to new, unseen data. The goal during training is to adjust the model's parameters to minimise this loss—often using an algorithm called Gradient Descent.

In short, loss functions steer the learning process, helping the model become more reliable and accurate.

In deep learning, choosing the right loss function depends on the type of task you're solving. For regression problems, where we predict continuous values, we often use mean squared error or mean absolute error.
Mean squared error calculates the average of squared differences between predicted and actual values—this makes it more sensitive to large errors.
Mean absolute error, on the other hand, averages absolute differences and is less affected by outliers.

For classification problems, where we predict categories, we use different loss functions.
Binary cross-entropy is used when there are just two possible classes. It measures the difference between predicted probabilities and actual binary labels.
Categorical cross-entropy works for multi-class problems, comparing the predicted probabilities to the true class.
And hinge loss, often used in support vector machines, penalises predictions that are too close to the incorrect class.

By matching the loss function to the type of problem, we can improve both learning efficiency and model accuracy.

Mean Squared Error, or M-S-E, is one of the most common loss functions used in regression tasks. It calculates the average of the squared differences between predicted and actual values. The formula is: **M-S-E equals one divided by n, multiplied by the sum of y-i minus y-hat-i squared**. Here, n is the number of observations, y-i is the actual value, and y-hat-i is the predicted value.

M-S-E penalises larger errors more than smaller ones because squaring amplifies the differences. This makes it especially sensitive to outliers. It's commonly used when the goal is to predict continuous values, like house prices or temperature. Although M-S-E provides a clear measure of prediction accuracy, its sensitivity to extreme values means we must use it carefully. Minimising M-S-E helps improve how accurately our model learns during training.

Mean Absolute Error, or M-A-E, is a common loss function used in regression problems. It measures the average absolute difference between the predicted values and the actual values. The formula is: M.A.E. equals one divided by *n*, multiplied by the sum of the absolute difference between each actual value, *y i*, and the predicted value, *y hat i*.

Unlike Mean Squared Error, which penalises larger errors more heavily, M-A-E treats all errors equally. This makes it less sensitive to outliers and a useful choice when working with datasets containing extreme values. It's often used to evaluate model performance in tasks like forecasting sales or predicting continuous outcomes. By reducing the M.A.E. during training, we can improve how accurately a model predicts new data.

Binary Cross-Entropy Loss, or B-C-E, is designed for binary classification tasks where the output is a probability between zero and one. It helps the model learn by comparing how close its predicted probabilities are to the actual labels, which are either zero or one. The formula looks like this:
BCE equals negative one over n, multiplied by the sum from i equals one to n, of y-i times log of y hat-i, plus one minus y-i times log of one minus y hat-i.
This formula penalises incorrect predictions based on how far they are from the actual values. A high-confidence wrong prediction leads to a bigger penalty than a low-confidence one. B-C-E is widely used in tasks like spam detection, where accurate binary decisions matter.

Categorical Cross-Entropy Loss, or C-C-E, is designed for multi-class classification problems. In these tasks, the model predicts a probability for each class, and C-C-E evaluates how close those predictions are to the actual labels.

The formula is:
C-C-E equals negative sum over C of y-i times log of y-hat-i.

Here, *C* is the number of classes, *y-i* is the actual label (zero or one), and *y-hat-i* is the predicted probability.

This loss function penalises predictions that are far from the actual class. The larger the error in the probability, the bigger the penalty. C-C-E is widely used in image classification tasks, where the model has to correctly label images across multiple classes.

Hinge Loss is a specialised loss function often used in maximum-margin classification, like Support Vector Machines, or S-V-Ms. Its main goal is to maximise the margin between different classes by focusing on the boundary.
The formula is:
Hinge Loss equals one divided by n, multiplied by the sum from i equals one to n, of max of zero comma one minus y-i times y-hat-i.
Here, n is the number of samples, y-i is the actual label, either plus one or minus one, and y-hat-i is the predicted value.
This loss penalises predictions that are either incorrect or too close to the decision boundary. A wider margin means the model is more confident and generalises better to new data.
It's especially useful in classification tasks where clear separation between classes is important.

Different loss functions are designed for different types of machine learning problems, and each has its own strengths. For regression tasks, Mean Squared Error—or M-S-E—penalises larger errors, while Mean Absolute Error, or M-A-E, treats all errors equally. Binary Cross-Entropy Loss, B-C-E, is used in binary classification and penalises wrong predictions based on probability. Categorical Cross-Entropy, C-C-E, works well for multi-class classification by evaluating how well predicted probabilities match actual classes. Hinge Loss is used in support vector machines and focuses on the margin between classes. Kullback-Leibler Divergence, or K-L Divergence, is mainly used in generative models and measures how different one probability distribution is from another. Choosing the right loss function is key to helping your model learn effectively and perform better on real data.

Think your model's doing great? Let the loss function be the judge. Let's see which one

keeps your predictions on point!

Video 4 - Gradient Descent

What if finding the best solution was just a matter of taking tiny steps downhill? That's Gradient Descent—like a treasure hunt where each clue gets you closer to machine learning gold.

Gradient Descent is a powerful optimisation tool in machine learning. It updates model parameters step by step, following the negative gradient, to reduce errors and improve predictions. This method helps models find the minimum of a function efficiently. From training neural networks to fine-tuning algorithms, Gradient Descent plays a key role in improving model accuracy.

In optimisation, the goal is to find the best possible solution by adjusting certain parameters. This could mean reducing costs, improving a model's accuracy, or balancing trade-offs. An objective function defines what we're trying to minimise or maximise—like cost, error, or time.

However, optimisation isn't always smooth. Some solutions may get stuck in local minima—points that seem optimal but aren't the best overall. Gradient Descent helps us move past these points and reach the global minimum, the most optimal solution.

Real-world problems often include constraints, such as time, budget limits, or compliance rules. Gradient Descent can adapt to these restrictions and still help find practical, workable solutions.

Gradient Descent begins by setting the model's starting values. These initial values matter—they can affect how fast and where the model converges. Next, the algorithm calculates the gradient, which shows the direction in which the loss increases most. Finally, the model updates its parameters by stepping in the opposite direction, reducing the loss and improving its performance with each iteration.

There are three main variants of Gradient Descent, each with its strengths and trade-offs. **Batch Gradient Descent** uses the entire dataset to compute the gradient at each step. This approach is accurate, but can be slow and computationally expensive for large datasets. **Stochastic Gradient Descent**, or **S-G-D**, updates the model using just one data point at a time. It's faster and more efficient, but can be noisy and unstable. However, that same randomness can help it escape local minima and explore the solution space more effectively. **Mini-Batch Gradient Descent** strikes a balance between the two. It processes a small group of data points at each step, making it both efficient and stable—ideal for most real-world applications.

Gradient Descent is a powerful optimisation tool, but it comes with challenges.

In deep networks, gradients can either vanish—becoming too small—or explode—becoming too large. Vanishing gradients slow down learning or stop it completely. Exploding gradients lead to unstable updates. To fix this, we use techniques like batch normalisation or gradient clipping.

Another issue is saddle points. These are flat regions where the gradient is zero, but the model hasn't reached the minimum. Gradient Descent can get stuck here. Momentum or adaptive methods like Adam help the model keep moving forward.

Finally, there's the learning rate. If it's too high, the model overshoots the minimum. If it's too low, training becomes painfully slow. Finding the right balance is key. Using learning rate schedules or adaptive algorithms helps improve training efficiency.

Adaptive optimisation techniques improve how Gradient Descent learns, especially in complex or unstable training environments. Let's start with **Momentum**. Momentum-based methods, such as Nesterov Accelerated Gradient, use the previous gradients to guide the current update. This helps the model move smoothly through the optimisation landscape, accelerating convergence and helping avoid local minima and saddle points.

Next is **Adam**, short for Adaptive Moment Estimation. Adam dynamically adjusts the learning rate for each parameter based on both the mean and variance of past gradients. This makes it robust in noisy environments and effective across a wide range of neural network architectures. Adam combines the strengths of both Momentum and R-M-S-Prop, offering stable and efficient training.

R-M-S-Prop, or Root Mean Square Propagation, modifies the learning rate using the running average of squared gradients. This helps balance the updates across parameters, especially when gradients vary in scale. By reducing the learning rate for frequently updated parameters, R-M-S-Prop handles non-stationary objectives well and improves training in deep networks.

Lastly, **Adagrad** adjusts learning rates based on historical gradients. It increases the step size for rarely updated parameters while reducing it for frequently updated ones. This is particularly helpful for sparse data or features. However, a key limitation of Adagrad is that the learning rate can shrink too quickly, slowing convergence in the later stages of training.

Each of these methods enhances Gradient Descent's ability to adapt and converge, making them essential tools for modern machine learning.

In optimisation, it's important to know when to stop training. We use specific criteria to decide this.

One method is based on **absolute or relative tolerance**. If the change in the objective function or parameter values becomes very small—below a set threshold—it means the model has likely reached a satisfactory level of convergence. Setting the right tolerance helps avoid unnecessary computations while maintaining accuracy.

Another common approach is to set a limit on the **maximum number of iterations**. This prevents the algorithm from running endlessly. While this limit may stop training before perfect convergence, it helps manage time and resources, especially when working with large models or datasets.

We can also monitor the **gradient norm**, which measures the size of the gradients. When the norm falls below a certain value, it suggests that further updates would bring minimal change. This signals that the solution has likely converged to a stationary point.

Using these stopping criteria helps balance efficiency and accuracy during the training process.

Gradient Descent is a foundational tool across several industries due to its ability to optimise performance through iterative learning.
In **machine learning**, it supports key algorithms such as linear regression, logistic regression, and neural networks. These models use Gradient Descent to adjust parameters and minimise the loss function, improving accuracy in tasks like image recognition and natural language processing.
In **robotics and control**, Gradient Descent is used to optimise control parameters, enabling robots to make better decisions and perform complex tasks like navigation, path planning, and motion coordination. This enhances both the autonomy and reliability of robotic systems, from industrial automation to assistive technologies.
In **signal processing**, it is applied to methods like adaptive filtering and beamforming. Gradient Descent helps optimise system parameters, improving clarity and precision in

applications such as noise reduction, signal enhancement, and audio reception.
In **finance and economics**, Gradient Descent helps model financial systems, adjust parameters for investment portfolios, and forecast market trends. It supports data-driven decision-making by maximising returns, minimising risks, and helping analysts navigate market complexities effectively.

Think Gradient Descent is just maths on paper? Think again—it's behind smarter robots, sharper forecasts, and better apps. Now it’s your turn: Try building a simple model and watch Gradient Descent in action!

Video 5 – Neural Networks- Demo

Hey learners, now that you have gone through your neural network basic concepts, let's understand a few more concepts and refresh some of them that we have already discussed. As we begin exploring how neural networks learn, it's important to understand that not all learning is the same. Neural networks are incredibly flexible and can adapt to different types of learning paradigms based on the nature of the data available and the goal of the task.

Neural networks can be applied across multiple learning foundations, supervised, unsupervised and reinforcement learning. The foundation selected depends largely on how much we know about the data we are working with. For instance, do we have labelled outputs or are we trying to find patterns without any guidance? The type of learning directly influences how data is used in the training process.

In supervised learning, data includes both input and output labels. In unsupervised learning, only inputs are available. In reinforcement learning, data evolves over time through interactions with an environment.

Each learning paradigm requires the neural network to adopt its architecture and training strategy. For example, a network trained to classify images might look very different from one trained to play chess or generate text. Understanding these variations lays the groundwork for mastering when and how to use neural networks effectively.

Let's begin with supervised learning, the most commonly used paradigm in neural networks. In supervised learning, as we have discussed earlier, we train the model using labelled data. That means for every input, we already know the correct output.

The goal of the model is to learn a mapping from inputs to these known outputs. Over time, the model improves its predictions by minimising the difference between its predictions and the true labels. There are two primary types of tasks in supervised learning, which again we have discussed earlier.

This is just to refresh. First, classification, where the model assigns input to its one of several categories, like classifying an image as a cat or a dog. Second, regression, where the model predicts continuous values, such as estimating house prices based on the area and location.

Let's look at a few real-world examples. Image classification, distinguishing between cats and dogs, or could be facial recognition, or could be identifying type of vehicle, or from a surveillance perspective, identifying if any issue is there in marketplaces or any kind of public places. It could be house price prediction, estimating the value of a property using past data, or spam detection, identifying whether an email is spam or not.

If you closely look at it, when we are talking about type of problem that we are solving using neural networks, they are the same problem that we have discussed under the hood of machine learning algorithms. The problem remains same, it's just that we have a

different approach. Think that way.

So, supervised learning is a type of learning that neural network supports, and it's quite powerful when you have labelled data, but not all problems come with labelled example. Let's now move to unsupervised learning, where the model has to make sense of the data on its own. Let's talk about unsupervised learning, where neural networks operate without labelled outputs.

In unsupervised learning, we provide the model with input data, but we don't tell it what the output should be. The model is left to discover patterns and structure on its own. It learns relationships and grouping in the data without external guidance.

Some common tasks in unsupervised learning include clustering, where the model groups similar data points together, dimensionality reduction, which simplifies large data sets by identifying the most important features, and autoencoders, which is type of neural network which supports your unsupervised learning or comes under supervised learning, which compresses and reconstructs data to learn efficient representations. Some real-world examples of unsupervised learning include customer segmentation, where businesses group customers based on behaviour, anomaly detection, such as identifying fraudulent transactions, and topic modelling, where we uncover themes in large collection of documents. Unsupervised learning is powerful when you don't have labelled data, but what if the learning happens over time through interactions? That's where reinforcement learning comes in.

Let's explore that next. We now come to the third major type of learning in neural networks, reinforcement learning. This learning paradigm is inspired by how humans and animals learn through interaction with their environment.

At the centre of the reinforcement learning is the agent. This is the learner or decision maker. The agent takes a cross in an environment in order to achieve a goal.

The environment is the world the agent interacts with. It provides context and response to the agent's actions. For example, in a game, the board or a screen is the environment.

So, when we play some kind of game, we are the agent or we are controlling the agent, which is interacting with the environment. And environment is your different stages that you play in the game. Then you have reward.

Each time the agent takes an action, it receives a reward, a signal that tells the agent whether action was good or bad. The goal is to maximise the total reward over time. Then you have policy.

The agent follows a policy, which is a strategy for choosing actions. The policy evolves as the agent learns from the rewards it receives. This creates a continuous learning loop.

The agent takes an action, the environment responds, the agent receives a reward, and then updates its policy to do better next time. Reinforcement learning powers some of the most impressive AI systems today, from self-driving cars to game-playing bots like AlphaGo. Let's now summarise what we have learned so far about different types of learning used in neural networks.

Supervised learning. This approach uses label data where each input is paired with the correct output. The task is usually prediction, such as classifying images or forecasting prices.

Unsupervised learning. We work with unlabelled data. The model isn't told what the output should be.

The goal is to discover hidden structure or patterns in the data, such as grouping similar customers or compressing data with auto-encoders. Reinforcement learning. The model learns through interactions with an environment.

It uses reward signals to evaluate actions and improve its decision-making strategy over time. You will see this used in robotics, game AI, and autonomous systems. Each learning type addresses different problems and requires different data and strategies.

Understanding when to use each is key to design effective neural network solutions. Now that we have explored different types of learning, let's discuss a critical challenge in training neural networks. Overfitting, which again we have discussed but now we are contextualising it for neural networks.

Overfitting happens when a model becomes too good at learning the training data. It performs exceptionally well during training but fails to generalise the new unseen data. This means it has essentially memorised the training example rather than learn underlying patterns.

So why are neural networks especially prone to overfitting? It's because they have a high capacity of memorisation. With many layers and parameters, neural networks can learn very complex functions, even noise in the data. This power is both a strength and a vulnerability.

The more flexible the model, the greater the risk of overfitting unless we apply proper controls which we will touch on later. To build effective neural networks, it's crucial to strike the right balance between complexity and simplicity. That's where this comparison comes in overfitting versus underfitting.

On the left, we have overfitting. This occurs when the model becomes too complex. It memorises the training data including noise and performs poorly on new or unseen data.

Overfitted models have high accuracy during training but fail to generalise. On the other extreme, underfitting happens when the model is too simple. It fails to capture even the basic structure of patterns in the data.

As a result, it performs poorly both on training set and the test set. Think of this as a model that does not try hard enough. And then there is an ideal fit.

That's what we aim for. This is a model that captures meaningful patterns in the training data and performs well on unseen data. It generalises effectively which is the hallmark of a well-trained neural network.

The goal of model training is to reach this sweet spot. Not too simple, not too complex. That's what we aim for using techniques like regularisation, data augmentation, and careful tuning which we'll study later.

Now, let's understand what are the possible reasons of overfitting in neural networks. To avoid overfitting, we need to understand them. And neural networks are particularly vulnerable due to their flexibility and depth.

Here are the four most common causes of overfitting in neural networks. Too many parameters relative to data. When a model has a large number of trainable parameters but not enough data, it starts to memorise the training examples rather than learning general patterns.

This mismatch between model complexity and data size is one of the biggest drivers of overfitting. Long training duration. Too many epochs.

If you train a model for too long, even a good one can overfit. It continues to tweak parameters to perfectly fit the training data including noise. Instead of learning more meaningful representations, the model becomes too good at what it knows and poor at adopting to new information.

Smaller noisy datasets. When data is limited, the model doesn't get to see a wide enough variety of examples. Similarly, if the dataset is noisy with inconsistencies or errors, the model might learn the wrong thing.

Both situations reduce the model's ability to generalise. Lack of regularisation or data augmentation. If you don't actively apply methods that prevent memorisation like dropout, early stopping, or data augmentation, the model has nothing stopping it from fitting too closely to the training data.

To manage overfitting in neural network, we often apply regularisation techniques. These help the model learn meaningful patterns while resisting the urge to memorise noise or outliers in the training data. We have L1, L2 regularisation.

Some mathematical penalties added to the model's loss function so that it does not overfit. We have dropout, which is a popular technique where neurones are disabled randomly during training. This prevents specific neurones from becoming overly specialised or codependent.

You have early stopping. Sometimes the best model isn't the one that trained the longest. Early stopping monitors performance on validation data and halts training when performance starts to degrade.

These techniques are just starting points. We'll explore their implementation and tuning strategies in later sessions where we dive deeper into model optimisation. Next, we'll talk about hyperparameter in neural networks.

When training neural network, a critical piece of the puzzle lies in the setting the right hyperparameters. These are external configuration settings that are not learned from data but chosen before training begins. Configuration settings hyperparameters define how the learning process unfolds.

They include choices like learning rate, batch size, number of layers, number of neurones per layer, and the activation or loss function used. Each of these controls an aspect of model behaviour just like dials on a machine. Tuning requirements.

Hyperparameters don't come with universal defaults. They need careful tuning depending on your data set, model architecture, and problem complexity. Even small changes can lead to drastically different results.

Well-tuned hyperparameters can drastically improve model's accuracy and generalisation. On the flip side, poor choices like too high higher learning rate or too shallow a network can cause the model to underperform or never converge at all. Hyperparameters act like the control knobs of neural network experiment and mastering them take both experience and experimentation.

Let's now look at some of the commonly used hyperparameters and how they influence training outcomes. Some of the most common hyperparameters. Let's see how they influence the learning process.

Each of these plays a critical role in balancing training speed, model capacity, and generalisation. Learning rate defines the step size the model takes when updating weights. If it's too high, the model can become unstable and fail to converge.

If it's too slow, learning becomes painfully slow. Finding the sweet spot is essential. Epochs.

An epoch is one complete pass through the entire training data set. Too few epochs and the model might underfit. Too many and it risks overfitting.

We usually monitor validation loss to decide when to stop. Batch size. This refers to how many samples the model processes before updating weights.

Small batch size gives more updates but can be noisy. Larger batches are more stable but less responsive. It's a trade-off between accuracy and efficiency.

Then you have number of layers nodes. This determines the capacity of neural network. More layers or neurones give the model more power to learn complex patterns but too much capacity can again lead to overfitting if not regularised.

Tuning these hyperparameters properly can make the difference between a mediocre model and one that performs exceptionally well. Next, we'll shift gears and understand how neural network actually learns. Just an intuition behind it.

One of the most powerful ideas behind how neural network learn is back propagation. This process adjusts the network's weights in a way that minimises error and here is how it works step by step. First the input data flows forward through the network.

Each layer performs computations combining input with weights adding biases and applying activation functions until a final prediction is produced. Then error calculation happens. The model compares its prediction with the actual label.

The difference is measured using a loss function such as mean squared error or cross entropy which tells us how wrong the prediction was. Now the magic begins. The error is propagated backward through the network.

Each neurone looks at how much it contributed to the final error and calculates a signal indicating how it should change. This process flows from the output layer all the way back to the input. And finally the model uses these signals to update the weights making small adjustments in the direction that reduces error.

This is typically done using gradient descent or a similar optimisation algorithm. In simpler terms back propagation helps the model learn which part of the network are responsible for the error and fine-tunes them accordingly. Just like improving skill by figuring out what went wrong and practising that part.

Now let's discuss some common neural network use cases. Neural networks have transformed the way we solve real world problems across the wide range of industries. Let's walk through some of the most impactful use cases.

Computer vision. In computer vision neural networks especially convolutional neural networks are used for image classification, object detection, face recognition and even medical image analysis. Whether it's tagging friends in photos or diagnosis diseases from scans CNNs are at the core.

Natural language processing. In NLP neural network power text classification, language translation and sentiment analysis. A special type of neural network rectal neural networks and transformers like BERT and GPT help machines understand and generate human language.

Healthcare. Neural network assist with disease prediction, drug discovery and interpreting medical images. They can detect anomalies in x-rays or even predict disease outbreak from historical trends.

Finance. In the financial world neural networks are used for fraud detection and credit scoring and even stock trend forecasting. They can analyse vast amount of transactional or time series data far better than traditional models.

Recommender systems. The power recommender system as well. From recommending movies on Netflix to products on Amazon neural networks help personalise experience by analysing your preferences and behaviour.

As you can see neural networks aren't just theoretical, they're driving value across industries and these use cases will only continue to grow.